Practica V

Muñoz Trujillo Alexis Eduardo 324037924

Programa 1

Primero vamos a utilizar `Scanner` para recibir datos desde la terminal. Necesitaremos importar:
"import java.util.Scanner;"

1. **nextInt() lee el siguiente dato de la entrada y lo devuelve como un entero, hasNextInt() revisa si el siguiente dato puede interpretarse como un entero y devuelve true o false,**

2. **Si se utiliza nextInt() y el usuario escribe algo que no es un entero, se produce una excepcion y el programa puede detenerse,**

3. **hasNextInt() permite revisar el dato antes de leerlo, evitando que nextInt() intente convertir una entrada invalida, si devuelve false, se puede descartar el dato con next() y volver a solicitarlo,**

Estrategia del ejercicio 3: primero se solicita la cantidad de dardos, despues se utiliza `hasNextInt()` para comprobar que la entrada sea un entero, si no lo es, se descarta con `next()`, tambien se verifica que el numero sea positivo, cuando el numero ya es correcto el programa realiza la estimacion de π como en la practica anterior,

Programa 2

Estrategia para resolver el problema: se utiliza hasNextInt() para comprobar que el dato ingresado sea un entero, despues se verifica que no sea negativo, para realizar la conversion se obtiene el residuo de dividir el numero entre dos y se agrega al inicio de una cadena de texto, posteriormente se divide el numero entre dos hasta que llegue a cero, si el numero ingresado es cero, se asigna directamente "0" como resultado,

Para permitir varias conversiones, se agrega un ciclo while que se repite cuando el usuario responde s o S, cualquier otra respuesta termina el programa donde se valida usando el charAt() la respuesta del usuario

Programa 3

Registro de datos: primero se solicita el nombre del cliente y despues se piden el tipo, capital, tasa y plazo, para validar los datos numericos se utiliza hasNextDouble() o hasNextInt() antes de leerlos, tambien se comprueba que el capital y el plazo sean positivos y que la tasa no sea negativa,

Menu: se utiliza un ciclo while para mostrar el menu repetidamente, dependiendo de la opcion seleccionada se realizan los calculos correspondientes, la opcion 4 permite modificar el capital sin volver a registrar los demas datos, las opciones 1, 2 y 3 utilizan el capital actual y la tasa ajustada segun el tipo de cliente,

Contador de operaciones y errores: se utilizan dos variables enteras, operaciones aumenta cuando se selecciona una opcion valida del 1 al 4, mientras que errores aumenta cuando se ingresa un dato que no corresponde a ninguna opcion del menu, la opcion 0 no aumenta el contador de operaciones, despues de cada seleccion se revisa si se alcanzo alguno de los limites,

Finalizacion: el programa termina cuando el usuario selecciona 0, cuando completa cinco operaciones efectivas o cuando comete tres errores en el menu, al terminar se muestra un resumen con el nombre, las operaciones realizadas, los errores, el capital final y el motivo de finalizacion,

Conclusion

1. **En general me parecen buenos ejercicios para familiarizarse con la clase Scanner, pero en mi caso no tuve muchas complicaciones ya que ya tengo experiencia programando e incluso en actividades similares cuando aprendi a programar en pascal**

2. **Tambien considero positivo que se usen los mismos ejercicios pero ahora con data dinamica porque algunos se preguntaban como podrian ser mas flexibles los programas que hemos hecho**

3. **La parte que mas se repite y logicamente es el while con el scanner para pedir a los usuarios que ingresen la data y en general las validaciones de la misma**

***Java Scanner NextInt() Method*. (s. f.). W3Schools.**

https://www.w3schools.com/java/ref_scanner_nextint.asp

***Java Scanner hasNextInt() method*. (s. f.). W3Schools.**

https://www.w3schools.com/java/ref_scanner_hasnextint.asp

***Java Scanner next() Method*. (s. f.). W3Schools.**

https://www.w3schools.com/java/ref_scanner_next.asp

***Java String charAt() method*. (s. f.). W3Schools.**

https://www.w3schools.com/java/ref_string_charat.asp